

智能巡检车 PRO 版实验手册

机器人目标检测模型应用

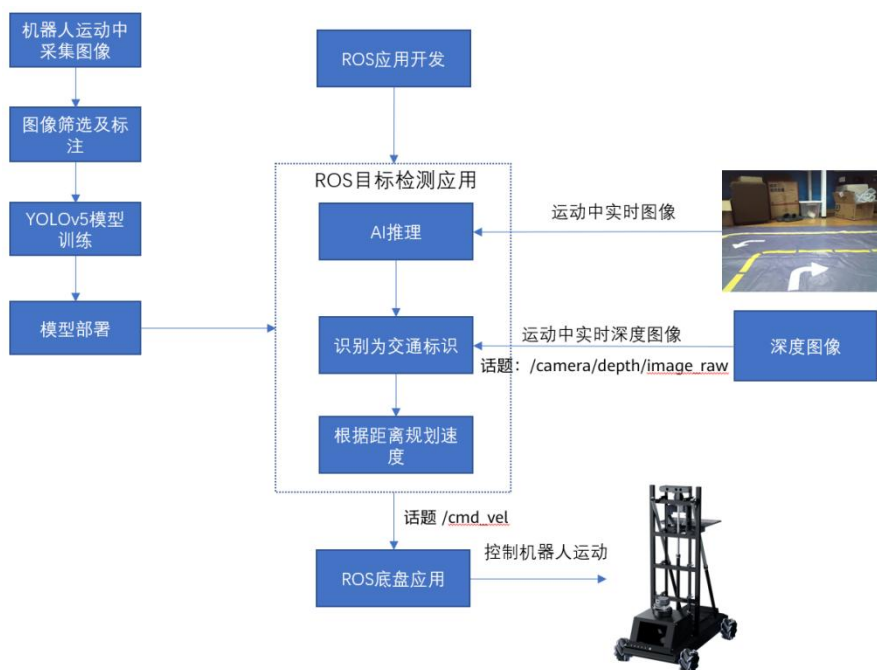
目录

1 实验背景	2
2 实验原理	2
3 实验目的	3
4 实验环境	3
5 实验流程	4
6 实验任务	5
6.1 任务一 准备边缘端基础环境	5
6.2 任务二 使用 YOLOv5 模型进行推理	9
6.3 任务三 通过网络调用 YOLOv5 模型推理	12
6.4 任务四 构建 ROS 目标检测应用	23
7 实验结果	32
8 实验资源释放	33
9 实验小结	33

1

实验背景

在前面章节中，我们提到了机器人目标检测功能总体流程：



在前面的课程中，我们已经完成了数据采集、标注，模型训练，本节将构建 ROS 目标检测应用，在机器人主控 Jetson Orin Nano 主控板系统中部署 AI 推理应用，调用前面训练出来的 YOLOv5 模型用于交通标识的目标检测，并将结果应用于控制机器人的运动。

2

实验原理

1. 机器人采用 Jetson Orin Nano 作为主控板，有 20TOPS 的边缘端智能算力，我们将上一节训练出的 YOLOv5 模型部署在该硬件系统中，作为 AI 推理应用。
2. 机器人装配了奥比中光 Astra Pro Plus ROS 实感深度相机，可以获得 RGB 彩色图像

和深度图像。

3. ROS 目标检测应用将采集到的 RGB 图像实时推送给 AI 推理应用，获得目标对象的类别和位置。
4. ROS 目标检测应用根据目标对象的类别和位置，并结合深度图像的距离信息，规划机器人的运动，并发布相关话题数据/cmd_vel。
5. 机器人的底盘应用节点订阅了/cmd_vel 话题数据，在收到数据后，控制底盘的运动。

3

实验目的

1. 了解如何在 Jetson Orin Nano 硬件系统的 Ubuntu20.04 系统中部署 YOLOv5
2. 了解如何在 Jetson Orin Nano 边缘端设备上进行 YOLOv5 的 AI 推理。
3. 掌握如何开发 ROS 目标检测应用，可以获取 RGB 图像，并调用训练出的 YOLOv5 模型推理。
4. 掌握如何开发 ROS 目标检测应用，可以根据深度图像信息获得目标对象距离，并控制小车底盘运动。

4

实验环境

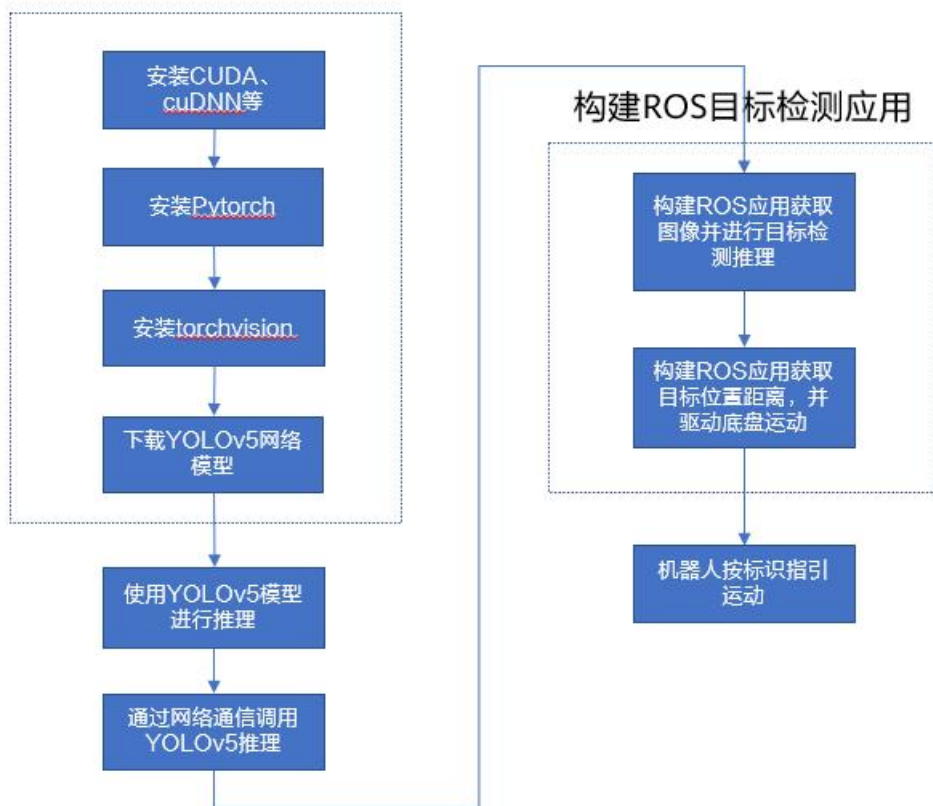
1. 硬件：iCar 智能小车，主控采用 Jetson Orin Nano。
2. 代码开发工具版本：VisualStudioCode 1.74.0 及以上。
3. 操作系统版本：Ubuntu20.04。
4. 开发语言：Python 3.8 及以上，C++ 11 及以上。
5. ROS 版本：ROS2。

5

实验流程

实验整体流程图如下所示：

安装边缘端推理基础环境



首先，我们假定用户已经准备好了 iCar 智能小车，机器人采用 Jetson Orin Nano 做主控板，运行 Ubuntu20.04 系统，安装了奥比中光 Astra Pro Plus 深度相机和 RPLidar 激光雷达。

在此基础上，还需要进行如下工作：

- (1) 在适配 Jetson Orin Nano 硬件的 Ubuntu20.04 系统中安装 Cuda、cuDNN、

Pytorch、torchvision，并下载 YOLOv5 网络框架代码，准备好基础环境。

- (2) 使用 YOLOv5 模型进行边缘端推理测试。
- (3) 通过网络通信调用 YOLOv5 模型进行边缘端推理。
- (4) 构建 ROS 目标检测应用，获取 RGB 图像并调用 YOLOv5 模型进行推理。
- (5) 构建 ROS 应用，根据推理结果的位置，从深度相机获取对应距离信息，并发布控制底盘运动的数据/cmd_vel。
- (6) 机器人将按交通标识指引在地图区域进行运动。

6

实验任务

6.1 任务一 准备边缘端基础环境

iCar 智能小车采用 Jetson Orin Nano 做主控板，安装的系统是 Ubuntu20.04，包含 Jetpack 5.1.1:

```
jetson@jetson-desktop:~/Desktop$ sudo jetson_release
Software part of jetson-stats 4.2.9 - (c) 2024, Raffaello Bonghi
Model: NVIDIA Orin Nano Developer Kit - Jetpack 5.1.1 [L4T 35.3.1]
NV Power Mode[0]: 10W
Serial Number: [XXX Show with: jetson_release -s XXX]
Hardware:
  Terminator : p3767-0004
  - Module: NVIDIA Jetson Orin Nano (4GB ram)
Platform:
  - Distribution: Ubuntu 20.04 focal
  - Release: 5.10.104-tegra
jtop:
  - Version: 4.2.9
  - Service: Active
Libraries:
  - CUDA: 11.4.315
  - cuDNN: 8.6.0.166
  - TensorRT: 8.5.2.2
  - VPI: Not installed
  - Vulkan: 1.3.204
  - OpenCV: 4.2.0 - with CUDA: NO
```

1. 配置需要安装的基础环境的源路径。

安装了 Jetpack 的操作系统中都会在 /etc/apt/source.list.d/ 中带有 nvidia-l4t-apt-source.list 文件，该文件中包含了需要安装基础环境的源路径，不用再单独去下载 CUDA、cuDNN、tensorRT 安装包，避免了版本配套或者兼容性问题，省去了很多麻烦。

```
$ vi /etc/apt/sources.list.d/nvidia-l4t-apt-source.list
# SPDX-FileCopyrightText: Copyright (c) 2019-2021 NVIDIA CORPORATION &
# AFFILIATES. All rights reserved.
# SPDX-License-Identifier: LicenseRef-NvidiaProprietary
#
# NVIDIA CORPORATION, its affiliates and licensors retain all intellectual
# property and proprietary rights in and to this material, related
# documentation and any modifications thereto. Any use, reproduction,
# disclosure or distribution of this material and related documentation
# without an express license agreement from NVIDIA CORPORATION or
# its affiliates is strictly prohibited.

deb https://repo.download.nvidia.com/jetson/common r35.4 main
deb https://repo.download.nvidia.com/jetson/<SOC> r35.3 main
```

这里只需将<SOC>修改为 t234 即可，对于 Jetson Orin 系列对应 t234，Jetson Xavier 系列对应 t194。

2. 更新源和安装。

```
$sudo apt update;
$sudo apt install cuda-11-4 libfreeimage-dev libcudnn8 libcudnn8-dev
libcudnn8-samples tensorrt tensorrt-dev tensorrt-libs python3-libnvinfer
python3-libnvinfer-dev uff-converter-tf onnx-graphsurgeon graphsurgeon-tf
```

3. 下载 Pytorch。

使用 pip3 下载软件包（-i 参数可以指定国内下载源进行加速）。

在如下网址可以下载 Pytorch 的 whl 安装文件：

<https://forums.developer.nvidia.com/t/pytorch-for-jetson/72048>

```
$sudo apt install python3-pip libopenblas-dev  
$python3 -m pip install aiohttp opencv-python scipy==1.5.3 -i  
https://pypi.tuna.tsinghua.edu.cn/simple  
$pip3 install ./torch-2.1.0a0+41361538.nv23.06-cp38-cp38-linux_aarch64.whl -i  
https://pypi.tuna.tsinghua.edu.cn/simple
```

4. 安装 torchvision。

Torchvision 是 Pytorch 中专门用来处理图像的库，包括数据集、模型、转换函数等。

安装编译 torchvision 所需依赖包：

```
$sudo apt install ffmpeg libavutil-dev libavcodec-dev libavformat-dev  
libavdevice-dev libavfilter-dev libswscale-dev libswresample-dev  
libswresample-dev libpostproc-dev libjpeg-dev libpng-dev
```

从本系列课程提供的资源目录下的“tools”路径下获取 torchvision 源码包

torchvision.tar 并解压缩：

```
$tar xvf torchvision.tar
```

进入 torchvision 目录并编译 torchvision

```
$cd torchvision  
$python3 setup.py install
```

利用编译成功的源码生成*.whl 文件，生成文件路径在/源码根目录/dist/

```
$python3 setup.py bdist_wheel
```

也可以采用 pip3 直接安装 0.16 版本的 torchvision：

```
$pip3 install torchvision==0.16 -i https://pypi.tuna.tsinghua.edu.cn/simple/
```

5. 验证 torch 和 torchvision 是否安装成功。

使用类似如下程序语句，如果输出计算结果则说明安装成功：

```
jetson@jetson:~$ python3  
Python 3.8.10 (default, Mar 13 2023, 10:26:41)  
[GCC 9.4.0] on linux  
Type "help", "copyright", "credits" or "license" for more information.  
>>> import torch  
>>> print(torch.__version__)  
2.1.0a0+41361538.nv23.06
```



```
>>> print('CUDA available:' + str(torch.cuda.is_available()))
CUDA available:True
>>> print('cuDNN version:' + str(torch.backends.cudnn.version()))
cuDNN version:8600
>>> a=torch.cuda.FloatTensor(2).zero_()
<stdin>:1: UserWarning: The torch.cuda.*DtypeTensor constructors are no longer
recommended. It's best to use methods such as torch.tensor(data, dtype=*,
device='cuda') to create tensors. (Triggered internally at
/opt/pytorch/pytorch/torch/csrc/tensor/python_tensor.cpp:82.)
>>> b=torch.randn(2).cuda()
>>> print(b)
tensor([-1.3097, -0.0221], device='cuda:0')
>>> print(str(b))
tensor([-1.3097, -0.0221], device='cuda:0')
>>> c=a+b
>>> print(c)
tensor([-1.3097, -0.0221], device='cuda:0')
>>> import torchvision
>>> print(torchvision.__version__)
0.16.2
```

6. 下载 YOLOv5 网络模型。

安装依赖环境：

```
pip3 install gitpython==3.1.30 numpy==1.22.2 setuptools==65.5.1 -i
https://pypi.tuna.tsinghua.edu.cn/simple
pip3 install onnx onnxruntime opencv-python ultralytics -i
https://pypi.tuna.tsinghua.edu.cn/simple
```

下载 YOLOv5 网络：

```
git clone https://github.com/ultralytics/yolov5.git
```

下载预训练模型：

```
wget https://github.com/ultralytics/yolov5/releases/download/v7.0/yolov5s.pt
```

说明：yolov5s.pt, yolov5m.pt, yolov5l.pt, yolov5x.pt, yolov5n.pt 是 YOLOV5 的不同变

体,表示不同大小和复杂性的预训练好的模型，可以用做自训练模型的基础。

从本系列课程提供的资源目录下的“tools”路径下获取上述 yolov5-7.0.zip 和 yolov5s.pt。

6.2 任务二 使用 YOLOv5 模型进行推理

1. 使用预训练模型 yolov5s.pt 进行推理

将获得的 yolov5s.pt 文件放到下载的 YOLOv5 网络的目录中，比如：

/home/jetson/yolov5-7.0。

在终端进入 YOLOv5 网络的目录中，并运行推理：

```
python3 detect.py --weights yolov5s.pt
YOLOv5 🚀 v7.0-231-gc2f131a Python-3.8.10 torch-2.1.0a0+41361538.nv23.06
CUDA:0 (Orin, 14533MiB)
Fusing layers...
YOLOv5s summary: 213 layers, 7225885 parameters, 0 gradients
image 1/2 /mnt/yolov5/data/images/bus.jpg: 640x480 4 persons, 1 bus, 211.0ms
image 2/2 /mnt/yolov5/data/images/zidane.jpg: 384x640 2 persons, 2 ties,
218.8ms
Speed: 3.3ms pre-process, 168.0ms inference, 10.5ms NMS per image at shape (1,
3, 640, 640)
Results saved to runs/detect/exp
```

可以看到，推理成功，结果保存在 runs/detect/exp 目录中。

```
jetson@jetson:~/yolov5-7.0/runs/detect/exp$ ls
zidane.jpg
jetson@jetson:~/yolov5-7.0/runs/detect/exp$ xdg-open zidane.jpg
```

可以看到，样例图片的推理结果：



预训练模型从该样例图片中识别到了两个人（类别为 person）和两个领结（类别为 tie），对识别结果进行了画框定位，并展示了置信度数据。

2. 使用自己训练的 YOLOv5 模型进行推理。

将 7.2 节中，我们自己训练 YOLOv5 模型的数据和模型结果，拷贝到 TRAFFIC 目录：

```
jetson@jetson-desktop:~/yolov5-7.0$ ls TRAFFIC/  
best.pt  classes.txt  labels.cache  test  train  val  voc_traffic.yaml
```

使用 tree 命令查看 TRAFFIC 目录：

```
jetson@jetson-desktop:~/yolov5-7.0$ tree -L 2 TRAFFIC/TRAFFIC/
.
├── best.pt
├── classes.txt
├── labels.cache
├── test
│   ├── images
│   └── labels
├── train
│   ├── images
│   ├── images (copy)
│   ├── labels
│   ├── labels.cache
│   └── labels (copy)
├── val
│   ├── images
│   ├── labels
│   └── labels.cache
└── voc_traffic.yaml

11 directories, 6 files
```

编辑 voc_traffic.yaml 配置文件:

```
jetson@jetson:~/yolov5-7.0$ vi TRAFFIC/voc_traffic.yaml
train: ./TRAFFIC/train/images
val: ./TRAFFIC/val/images
test: ./TRAFFIC/test/images
# number of classes
nc: 3
# class names
names: [ 'turnleft', 'turnright','turnaround']
```

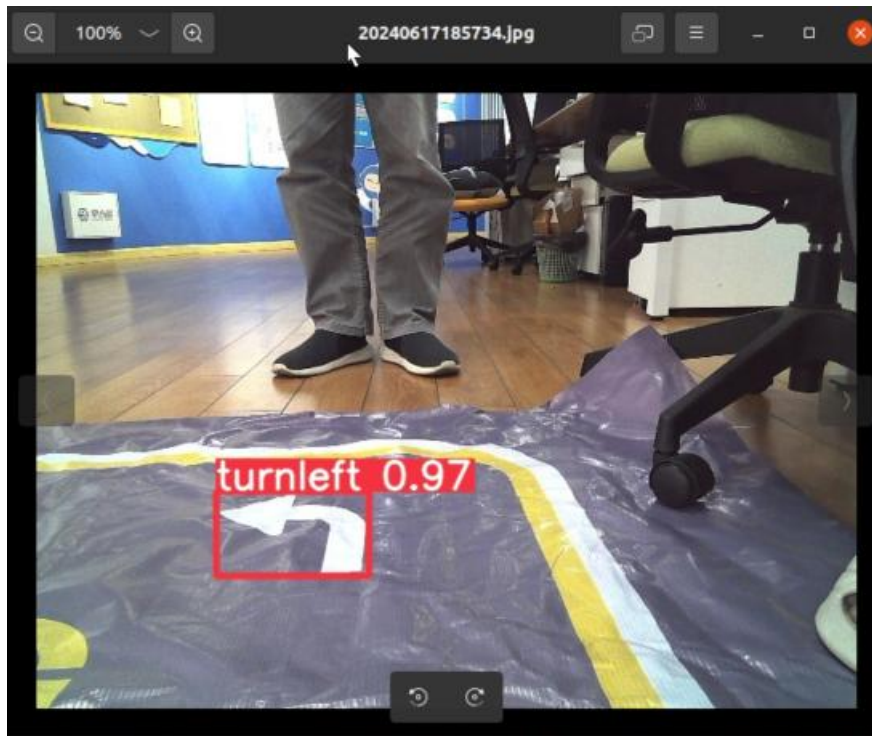
使用 TRAFFIC/best.pt 模型进行推理:

```
jetson@jetson-desktop:~/yolov5-7.0$ python3 detect.py --source TRAFFIC/test/images/20240617185734.jpg --weights TRAFFIC/best.pt
detect: weights=['TRAFFIC/best.pt'], source=TRAFFIC/test/images/20240617185734.jpg, data=data/coco128.yaml, imgsz=[640, 640], conf_thres=0.25, iou_thres=0.45, max_det=1000, device=, view_img=False, save_txt=False, save_conf=False, save_crop=False, nosave=False, classes=None, agnostic_nms=False, augment=False, visualize=False, update=False, project=runs/detect, name=exp, exist_ok=False, line_thickness=3, hide_labels=False, hide_conf=False, half=False, dnn=False, vid_stride=1
YOLOv5 2022-11-22 Python-3.8.10 torch-2.1.0a0+41361538.nv23.06 CUDA:0 (Orin, 3311MiB)

Fusing layers...
YOLOv5s summary: 224 layers, 7059304 parameters, 0 gradients
image 1/1 /home/jetson/yolov5-7.0/TRAFFIC/test/images/20240617185734.jpg: 480x640 1 turnleft, 216.7ms
Speed: 10.9ms pre-process, 216.7ms inference, 49.1ms NMS per image at shape (1, 3, 640, 640)
Results saved to runs/detect/exp6
```

推理结果保存在 runs/detect/exp6。

```
jetson@jetson:~/yolov5-7.0$ xdg-open runs/detect/exp6/20240617185734.jpg
```



可以看到，模型推理正确识别了图像中的交通转向标识类别（“turnleft”），对位置进行了画框，并标记了置信度。

6.3 任务三 通过网络调用 YOLOv5 模型推理

1. 原理介绍

我们将 YOLOv5 模型推理能力以 AI 推理服务端方式对外提供，采用 TCP 网络通信，启动服务监听，服务 IP：x.x.x.x，服务端口：port，可以在我们提供的测试代码中，根据实际情况修改。



(1)、客户端 (可以是 ROS 应用, 也可以是其他需要调用 AI 推理服务的程序) 通过 socket 通信向服务端的服务 IP 和服务端口发起链接, 通过该链路进行数据通信。

(2)、客户端程序将需要推理的图片进行编码, 并通过 socket 发给服务端。

(3)、服务端接收图片并解码, 然后在边缘端 AI 硬件基础上调用 YOLOv5 模型推理。

(4)、服务端将模型推理结果, 包括类别及位置信息, 编码, 并发送给客户端。

(5)、客户端解析结果, 获取目标类别及位置信息, 并将结果用于相关业务逻辑的处理, 比如: ROS 应用根据识别到的交通标识类别和位置, 来定位、决策机器人该如何行动。

2. 部署服务端测试程序

从本系列课程提供的资源目录下的“源码”路径下获取 aiserver.py、self_detect.py、client.py 三个文件。

将 aiserver.py 和 self_detect.py 部署在机器人主控边缘端系统的如下位置:


```
jetson@jetson-desktop: ~/yolov5-7.0
jetson@jetson-desktop:~/yolov5-7.0$ pwd
/home/jetson/yolov5-7.0
jetson@jetson-desktop:~/yolov5-7.0$ ls
aiserver.py      CONTRIBUTING.md  hubconf.py      README.md       self_detect.py  tutorial.ipynb
benchmarks.py   data             LICENSE         requirements.txt setup.cfg        utils
classify        detect.py        models          runs            TRAFFIC         val.py
client.py        export.py        __pycache__     segment         train.py
jetson@jetson-desktop:~/yolov5-7.0$
```

client.py 可以和 aiserver.py 部署在一起，也可以部署在不同的主机上。

这里为简化实验环境，我们将 client.py 和 aiserver.py 部署在一起。

self_detect.py 中定义了 YoloDetector 类，对 YOLOv5 模型推理进行了接口包装。

YoloDetector 类在对象初始化时，根据配置参数加载模型，后续提供 detect 接口，可以持续接收图片进行推理。该文件较长，可以自行打开，查看代码逻辑。

aiserver.py 的代码如下：

```
import cv2

import numpy as np

import socket

from self_detect import YoloDetector, parse_opt

import time

import sys

def receive_image(conn):

    # 接收图片大小

    data_len_str = conn.recv(16).decode().strip()

    data_len = int(data_len_str)
```

```
# 接收图片数据

data = b''

while len(data) < data_len:

    packet = conn.recv(min(data_len - len(data), 1024))

    if not packet:

        break

    data += packet


# 解码图片

decoded_data = np.frombuffer(data, dtype=np.uint8)

image = cv2.imdecode(decoded_data, cv2.IMREAD_COLOR)


# 显示图片

cv2.imshow('Received Image', image)

cv2.waitKey(1)

#cv2.destroyAllWindows()

return image


if __name__ == "__main__":

    # 服务器地址

    host='192.168.8.101'

    port=12345
```



```
print("Open server:{} : {}".format(host, port))

server_address = (host, port)

opt = parse_opt()

self_yolo = YoloDetector(**vars(opt))

font = cv2.FONT_HERSHEY_SIMPLEX

# 创建 socket 服务器

with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:

    s.settimeout(5)

    s.bind(server_address)

    s.listen(1)

    print(f"Server is listening for connections on {host}:{port}...")

    while True:

        try:

            conn, addr = s.accept()

            print(f"Connected by {addr}")

            with conn:

                while True:

                    try:

                        frame = receive_image(conn)

                        res_img, yolo_list = self_yolo.detect(frame)
```

```
        pixel_list    =    self_yolo.yolo_to_pixel(yolo_list,
res_img.shape[0], res_img.shape[1])

        i = 1

        msg=""

        for e in pixel_list:

            cv2.circle(frame,    (int(e[0]),    int(e[1])),    2,
(255,0,0), -1)

            cv2.putText(frame,    str(e[2]),    (int    (e[0])+5,
int(e[1])), font, 0.5, (0,0,255),2)

            cv2.putText(frame,    str(e[3]),    (int    (e[0])+5,
int(e[1])+10), font, 0.5, (0,0,255),2)

            if i > 1:

                msg=msg+';'

                msg = msg + str(e[0]) + ';' + str(e[1]) + ';' +
str(e[3]) + ';' + str(time.time())

                i=i+1

            print(msg)

            conn.send(msg.encode('utf-8'))

            cv2.imshow('result', frame)

            cv2.waitKey(1)

        except socket.timeout:

            print("Timeout occurred while waiting for data")

            continue

        except ConnectionResetError:

            print("Connection reset by peer")
```

```
        break

    except Exception as e:

        print(f"An error occurred:{e}")

        break

    except KeyboardInterrupt:

        print("Server stopped by user.")

        break

    except Exception as e:

        print(f"An error occurred: {e}")

        continue
```

该程序的代码逻辑非常简单清楚，它使用 YoloDetector 类对象作为推理的执行者，在程序启动时，根据指定的 IP 和端口开启 Socket 服务，接收客户端的连接。如果有客户端连接上来，它就从 socket 连接中接受客户端发来的图像编码数据，并解码，对解码后的图像调用 YoloDetector 对象进行 AI 推理，将推理结果画在图像上展示给用户看，并将结果信息拼接成字符串编码后发送给客户端。

我们使用如下指令来启动 AI 推理服务程序：

```
cd ~/yolov5-7.0

python3 aiserver.py
```

```
jetson@jetson-desktop: ~/yolov5-7.0
jetson@jetson-desktop:~/yolov5-7.0$ python3 aiserver.py
Open server:192.168.8.101 : 12345
self_detect: weights=/home/jetson/yolov5-7.0/TRAFFIC/best.pt, data=/home/jetson/yolov5-7.0/TRAFFIC/voc_traffic.yaml, imgsz=[640, 640], conf_thres=0.25, iou_thres=0.45, max_det=1000, device=, classes=None, agnostic_nms=False, line_thickness=3, hide_labels=False, hide_conf=False, half=False, dnn=False
YOLOv5 🚀 2022-11-22 Python-3.8.10 torch-2.1.0a0+41361538.nv23.06 CUDA:0 (Orin, 3311MiB)

Fusing layers...
YOLOv5s summary: 224 layers, 7059304 parameters, 0 gradients
Server is listening for connections on 192.168.8.101:12345...
```

从打屏信息可以看到，服务端已经在 IP 地址 192.168.8.101 上开启了 12345 端口的监听，等待客户端连接。

3. 启动客户端测试程序

客户端测试程序代码如下：

```
import cv2

import numpy as np

import socket

import time

import os

def send_image(image_path, s):

    # 读取图片

    image = cv2.imread(image_path)

    # 编码图片为 JPEG 格式

    _, encoded_image = cv2.imencode('.jpg', image, [cv2.IMWRITE_JPEG_QUALITY, 80])

    # 将 NumPy 数组转换为字节流

    data = encoded_image.tobytes()
```

```
# 发送图片大小

s.sendall(f"{len(data):<{16}}".encode())

# 发送图片数据

s.sendall(data)

reply = s.recv(128)

return reply

def get_images(directory):

    images=[]

    image_extensions = ['.jpg', '.jpeg', '.png', '.gif']

    for root, dirs, files in os.walk(directory):

        for file in files:

            file_path = os.path.join(root, file)

            if any(file.lower().endswith(ext) for ext in image_extensions):

                images.append(file_path)

    return images

if __name__ == "__main__":

    # 图片路径

    directory = 'TRAFFIC/test/images/'

    images = get_images(directory)

    # 服务器地址

    server_address = ('192.168.8.101', 12345)
```

```
with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:

    s.connect(server_address)

    while True:

        for image_path in images:

            result = send_image(image_path, s)

            print(result.decode('utf-8'))

            time.sleep(0.5)

        print("finish one cycle...")

        time.sleep(3)
```

代码逻辑也非常简单、清楚。首先，从 TRAFFIC/test/images 目录获得测试图片列表；然后尝试连接到服务端（IP 地址是 192.168.8.101，端口是 12345）；如果连接成功，则会将图片列表中的图片发送给服务端进行 AI 推理，并打印返回的推理结果；当所有图片都推理完成，短暂休息 3 秒钟，再次进入推理循环。

我们使用如下指令来启动 AI 推理服务程序：

```
cd ~/yolov5-7.0

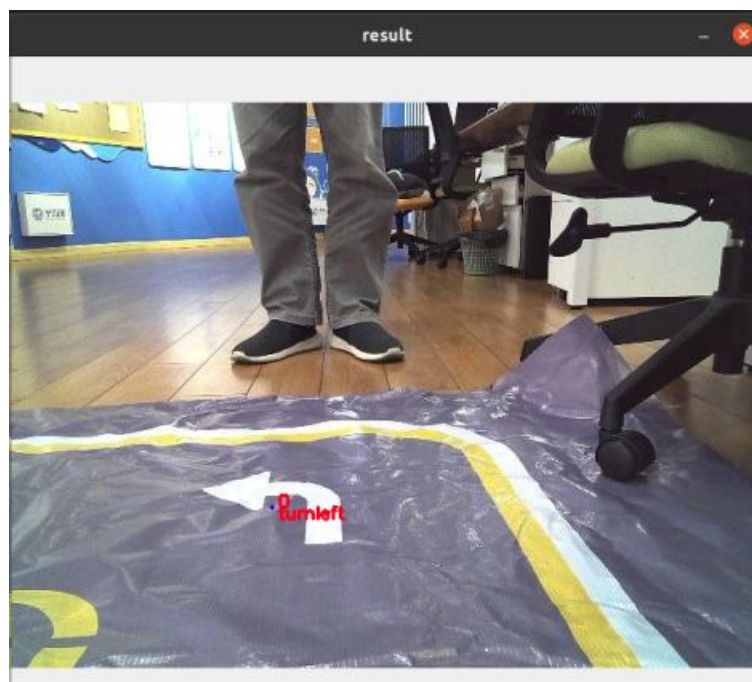
python3 client.py
```

在服务端的运行的终端窗口可以看到，服务端对于每张图像依次进行推理，并打印 xywh 信息及 x,y,label,time 信息：

```
jetson@jetson-desktop:~/yolov5-7.0$ python3 aiserver.py
Open server:192.168.8.101 : 12345
self_detect: weights=/home/jetson/yolov5-7.0/TRAFFIC/best.pt, data=/home/jetson/yolov5-7.0/TRAFFIC/voc_traffic.yaml, imgsz=[640, 640], conf_thres=0.25, iou_thres=0.45, max_det=1000, device=, classes=None, agnostic_nms=False, line_thickness=3, hide_labels=False, hide_conf=False, half=False, dnn=False
YOLOv5 🚀 2022-11-22 Python-3.8.10 torch-2.1.0a0+41361538.nv23.06 CUDA:0 (Orin, 3311MiB)

Fusing layers...
YOLOv5s summary: 224 layers, 7059304 parameters, 0 gradients
Server is listening for connections on 192.168.8.101:12345...
Connected by ('192.168.8.101', 45020)
xywh:[0.3187499940395355, 0.8166666626930237, 0.21562500298023224, 0.1875, 0]
203.99999618530273,391.99999809265137,turnleft,1731503954.1451254
xywh:[0.3648437559604645, 0.737500011920929, 0.19218750298023224, 0.1458333283662796, 0]
233.50000381469727,354.0000057220459,turnleft,1731503954.7740831
xywh:[0.21718749403953552, 0.8374999761581421, 0.22812500596046448, 0.19166666269302368, 0]
138.99999618530273,401.9999885559082,turnleft,1731503955.3740287
xywh:[0.3460937440395355, 0.715624988079071, 0.19218750298023224, 0.1354166716337204, 0]
221.49999618530273,343.4999942779541,turnleft,1731503955.9714963
xywh:[0.375, 0.6875, 0.1875, 0.12083332985639572, 0]
240.0,330.0,turnleft,1731503956.5759695
xywh:[0.38359373807907104, 0.6822916865348816, 0.18906250596046448, 0.11874999850988388, 0]
245.49999237060547,327.50000953674316,turnleft,1731503957.1733923
xywh:[0.4375, 0.78125, 0.23125000298023224, 0.1666666716337204, 0]
```

并针对每张图都展现一张推理结果图，图上标注了目标对象的画框中心点，和目标对象的类别（比如 0，对应了 turnleft 交通标识）：



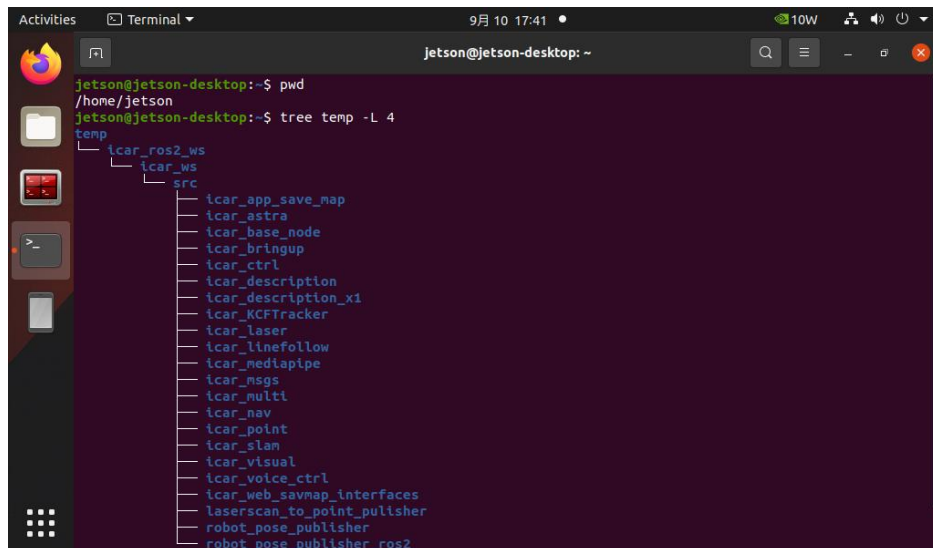
在客户端的终端窗口可以看到，对于每张图，都会收到服务端的推理结果，包含了目标对象的画框中心点坐标 x、y，目标对象的类别，时间戳：


```
jetson@jetson-desktop:~/yolov5-7.0$ python3 client.py
203.99999618530273,391.99999809265137,turnleft,1731503954.1451254
233.50000381469727,354.0000057220459,turnleft,1731503954.7740831
138.99999618530273,401.9999885559082,turnleft,1731503955.3740287
221.49999618530273,343.4999942779541,turnleft,1731503955.9714963
240.0,330.0,turnleft,1731503956.5759695
245.49999237060547,327.50000953674316,turnleft,1731503957.1733923
280.0,375.0,turnleft,1731503957.7713933
199.50000762939453,343.00000190734863,turnleft,1731503958.3726676
215.0,381.49998664855957,turnleft,1731503958.9585912
394.50000762939453,389.50000762939453,turnleft,1731503959.5644264
247.5,318.0000114440918,turnleft,1731503960.1629875
```

客户端可以利用这些信息去做自己的业务逻辑，比如 ROS 应用可以使用中心点坐标，到深度图像中匹配对应点的距离信息，并根据距离信息来控制机器人的前进、后退来进行对目标对象的跟随等。

6.4 任务四 构建 ROS 目标检测应用

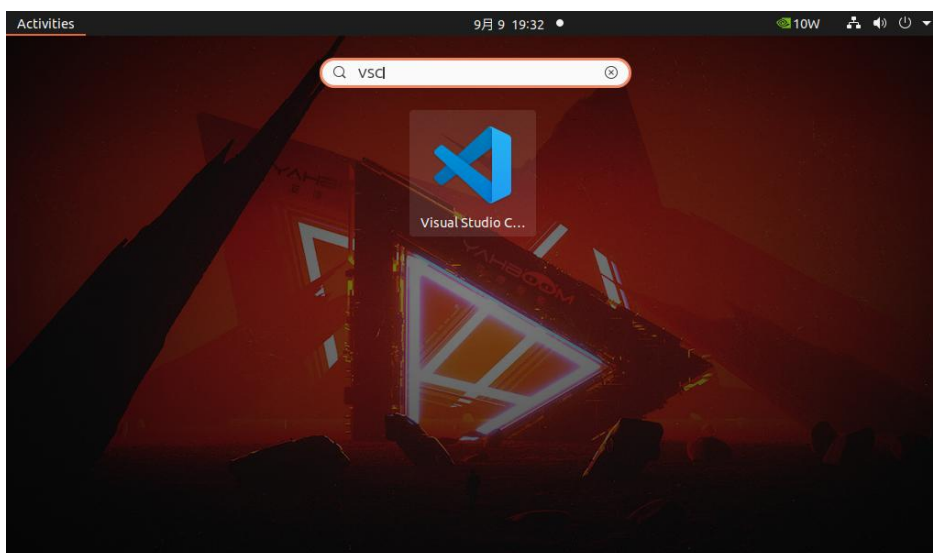
1. 在 iCar 智能小车上位机系统的如下路径中：~/temp，预置了相关功能代码：



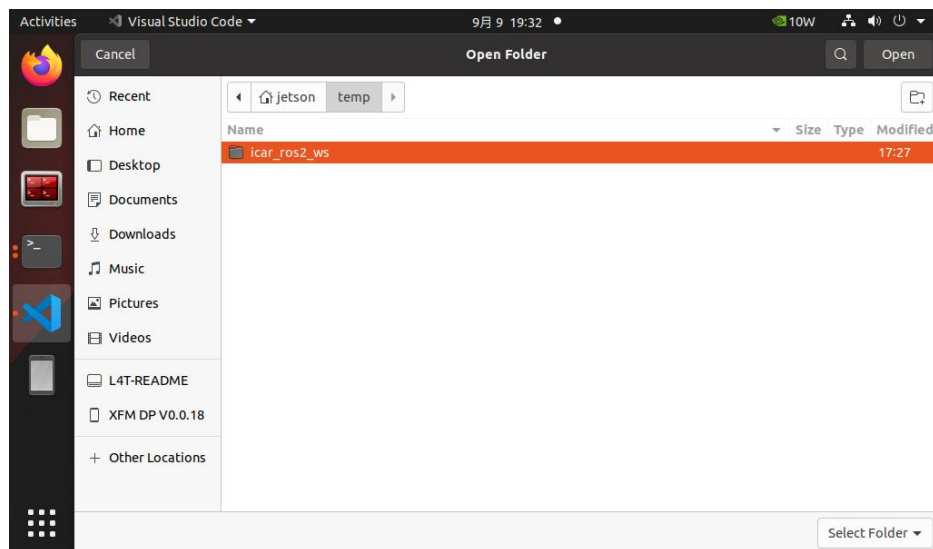
2. 点击左边栏的九宫格图标：



3. 搜索并启动已安装好的 VisualStudioCode 软件（以下简称 VSCode）：

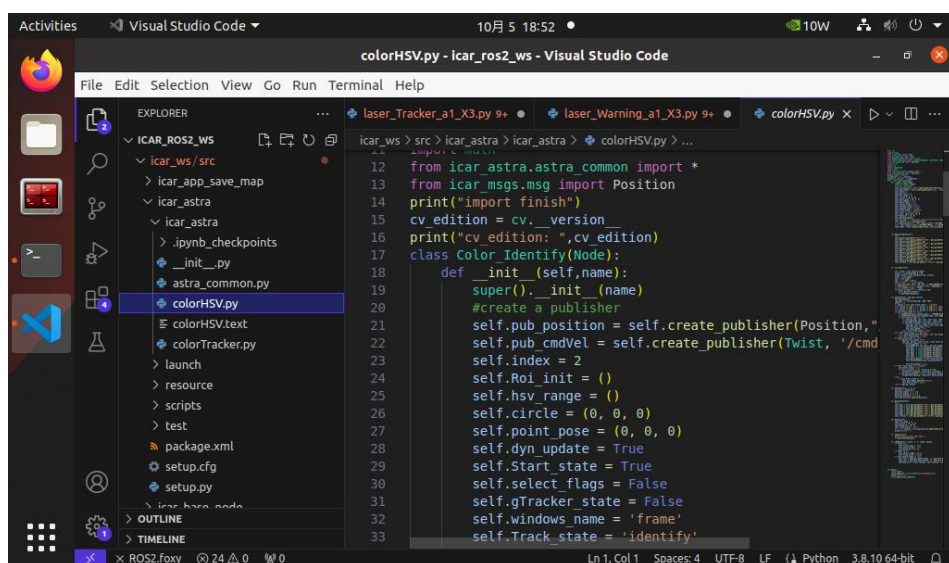


4. 使用 VSCode 打开步骤 1 中的代码：

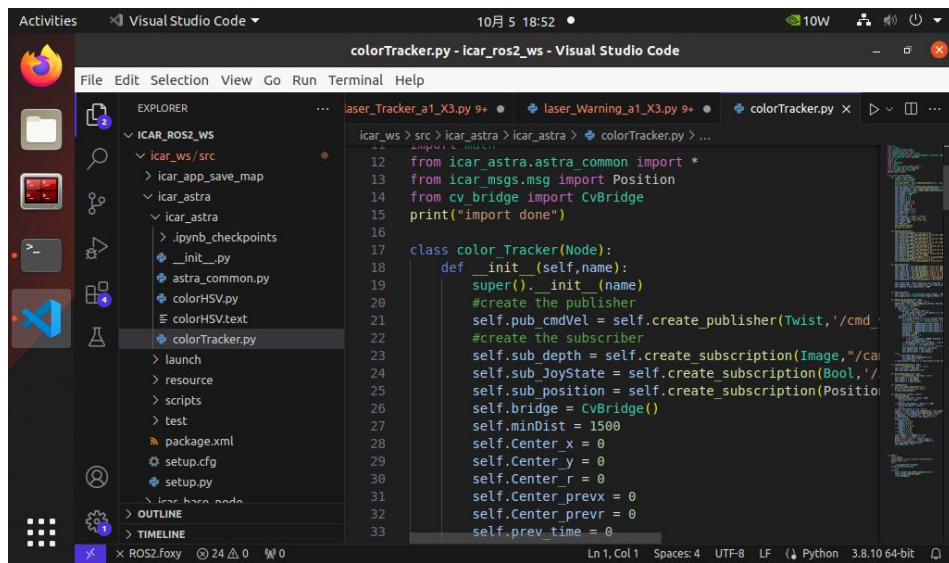


5. 可以查看颜色追踪或者底盘控制等 ros2 相关代码：

颜色对象识别的代码是下图的 colorHSV.py

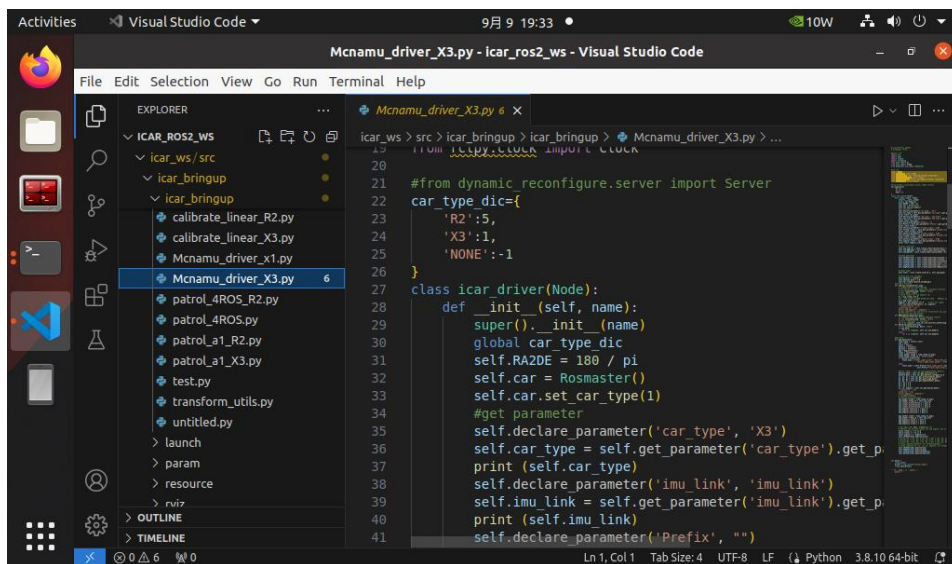


颜色对象跟踪的代码是下图的 colorTracker.py



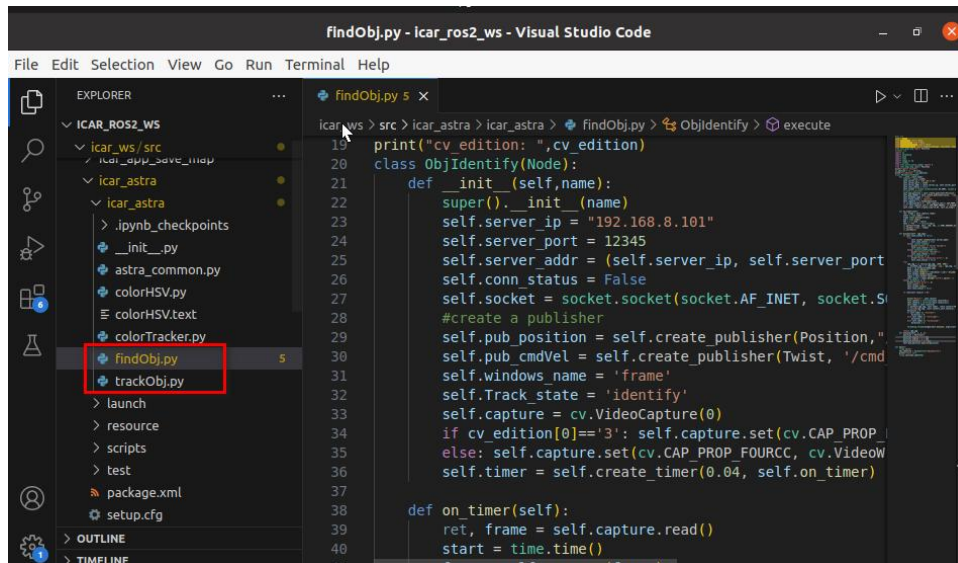
```
12 from icar_astra.astra_common import *
13 from icar_msgs.msg import Position
14 from cv_bridge import CvBridge
15 print("import done")
16
17 class color_Tracker(Node):
18     def __init__(self, name):
19         super().__init__(name)
20         #create the publisher
21         self.pub_cmdVel = self.create_publisher(Twist, '/cmd', 10)
22         #create the subscriber
23         self.sub_image = self.create_subscription(Image, '/camera/rgb/image_raw', self.image_callback, 10)
24         self.sub_JoyState = self.create_subscription(Bool, '/joy_state', self.joy_state_callback, 10)
25         self.sub_position = self.create_subscription(Position, '/position', self.position_callback, 10)
26         self.bridge = CvBridge()
27         self.minDist = 1500
28         self.Center_x = 0
29         self.Center_y = 0
30         self.Center_r = 0
31         self.Center_prev_x = 0
32         self.Center_prev_y = 0
33         self.prev_time = 0
```

底盘控制的代码是下图中的 Mcnamu_driver_X3.py



```
20 #from dynamic_reconfigure.server import Server
21 car_type_dic={
22     'R2':5,
23     'X3':1,
24     'NONE':-1
25 }
26
27 class icar_driver(Node):
28     def __init__(self, name):
29         super().__init__(name)
30         global car_type dic
31         self.RA2DE = 180 / pi
32         self.car = Rosmaster()
33         self.car.set_car_type(1)
34         #get parameter
35         self.declare_parameter('car_type', 'X3')
36         self.car_type = self.get_parameter('car_type').get_parameter_value().integer_value
37         print(self.car_type)
38         self.declare_parameter('imu_link', 'imu_link')
39         self.imu_link = self.get_parameter('imu_link').get_parameter_value().string_value
40         print(self.imu_link)
41         self.declare_parameter('Prefix', "")
```

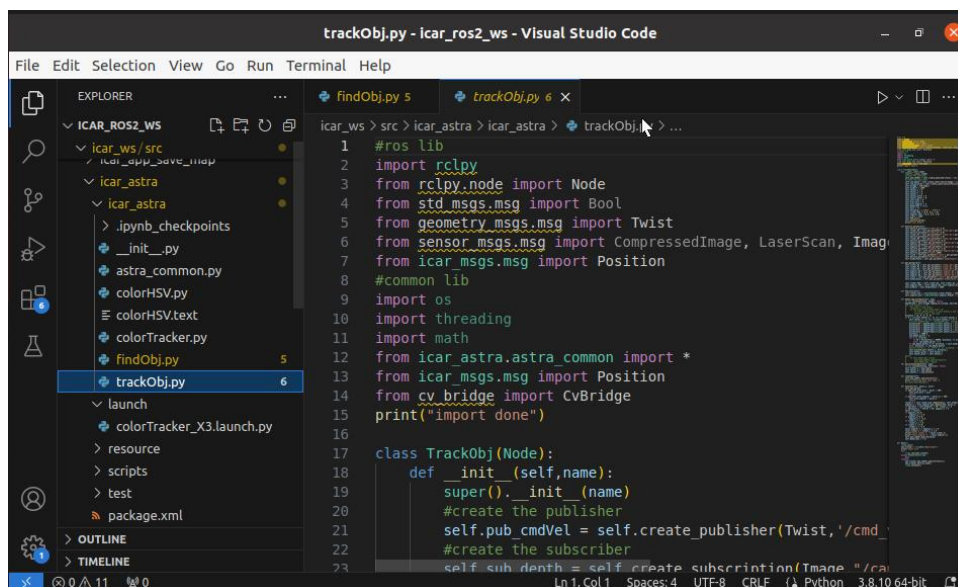
6. 我们新增目标对象检测应用节点等 ros2 相关代码：



```

19 print("cv edition: ",cv_edition)
20 class ObjIdentify(Node):
21     def __init__(self,name):
22         super().__init__(name)
23         self.server_ip = "192.168.8.101"
24         self.server_port = 12345
25         self.server_addr = (self.server_ip, self.server_port)
26         self.conn_status = False
27         self.socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
28         #create a publisher
29         self.pub_position = self.create_publisher(Position, "/cmd_vel", 10)
30         self.pub_cmdVel = self.create_publisher(Twist, '/cmd_vel', 10)
31         self.windows_name = 'frame'
32         self.Track_state = 'identify'
33         self.capture = cv.VideoCapture(0)
34         if cv_edition[0]=='3': self.capture.set(cv.CAP_PROP_FRAME_WIDTH, 640)
35         else: self.capture.set(cv.CAP_PROP_FRAME_WIDTH, 320)
36         self.timer = self.create_timer(0.04, self.on_timer)
37
38     def on_timer(self):
39         ret, frame = self.capture.read()
40         start = time.time()

```



```

1 #ros lib
2 import rclpy
3 from rclpy.node import Node
4 from std_msgs.msg import Bool
5 from geometry_msgs.msg import Twist
6 from sensor_msgs.msg import CompressedImage, LaserScan, Image
7 from icar_msgs.msg import Position
8 #common lib
9 import os
10 import threading
11 import math
12 from icar_astra.astra_common import *
13 from icar_msgs.msg import Position
14 from cv_bridge import CvBridge
15 print("import done")
16
17 class TrackObj(Node):
18     def __init__(self,name):
19         super().__init__(name)
20         #create the publisher
21         self.pub_cmdVel = self.create_publisher(Twist, '/cmd_vel', 10)
22         #create the subscriber
23         self.sub_depth = self.create_subscription(Image, "/camera/depth/image_raw", self.callback, 10)

```

其中 findObj.py 实现的功能是，从深度相机的 rgb 模块获取图像，并通过网络调用任务三中的 AI 推理服务来获得对图像的实时推理结果，如果有目标对象的推理结果，则通过 Position 消息格式以"/Current_point"话题来发布出来。

trackObj.py 实现的功能是，从订阅的深度图像话题"/camera/depth/image_raw"中获得深度信息，从订阅的"/Current_point"话题来获得目标对象的位置信息，将两者进行结合，以获得目标对象的距离信息，并据此发布话题"/cmd_vel"以调节底盘的运动，实现对目标对象的跟踪。

其中目标对象可以是如下三种图形之一(也可根据 7.2 节内容自行训练对特定目标进行检测的模型) :



代码行数较多, 可以从本系列课程提供的资源目录下的“源码”路径下直接获取。

7. 将以上代码更新到 docker 中的 ros2 代码工程中进行编译构建

代码需上传至 docker 中的 ros2 代码工程中进行替换, 两者的对应关系如下:

VSCode 中的代码, 在承载系统的宿主机如下目录:

```

root@jetson-desktop: ~/icar_ros2_ws/icar_ws/src
jetson@jetson-desktop:~$ docker exec -it e49 /bin/bash
Error response from daemon: Container e49c1206ae0f7d19051bed4745e10616015c56b4378fdcd4815982cbb548c is not running
jetson@jetson-desktop:~$ docker start e49
e49
jetson@jetson-desktop:~$ docker exec -it e49 /bin/bash
-----
ROS_DOMAIN_ID: 32
my_robot_type: x3 | my_lidar: a1 | my_camera: astraplus
-----
root@jetson-desktop:/# pwd
/
root@jetson-desktop:/# ls
CMakeLists.txt  README.md  core  etc  log  opt  root  sbins  sys  var
CMakeModules  bin  dev  home  media  package.xml  ros_entrypoint.sh  src  tmp
LICENSE  boot  docker  lib  mnt  proc  run  srv  usr
root@jetson-desktop:/# src
root@jetson-desktop:~/icar_ros2_ws/icar_ws/src# pwd
/root/icar_ros2_ws/icar_ws/src
root@jetson-desktop:~/icar_ros2_ws/icar_ws/src# ls
build          icar_description  icar_nav          laserscan_to_point_publisher
icar_KCFTracker  icar_description_x1  icar_point        log
icar_app_save_map  icar_laser         icar_slam         robot_pose_publisher
icar_astra        icar_linefollow    icar_visual       robot_pose_publisher_ros2
icar_base_node    icar_mediapipe     icar_voice_ctrl
icar_bringup      icar_msgs          icar_web_savmap_interfaces
icar_ctrl         icar_multi         install
root@jetson-desktop:~/icar_ros2_ws/icar_ws/src#

```

对应的在 docker 中的 ros 代码路径如下:

```

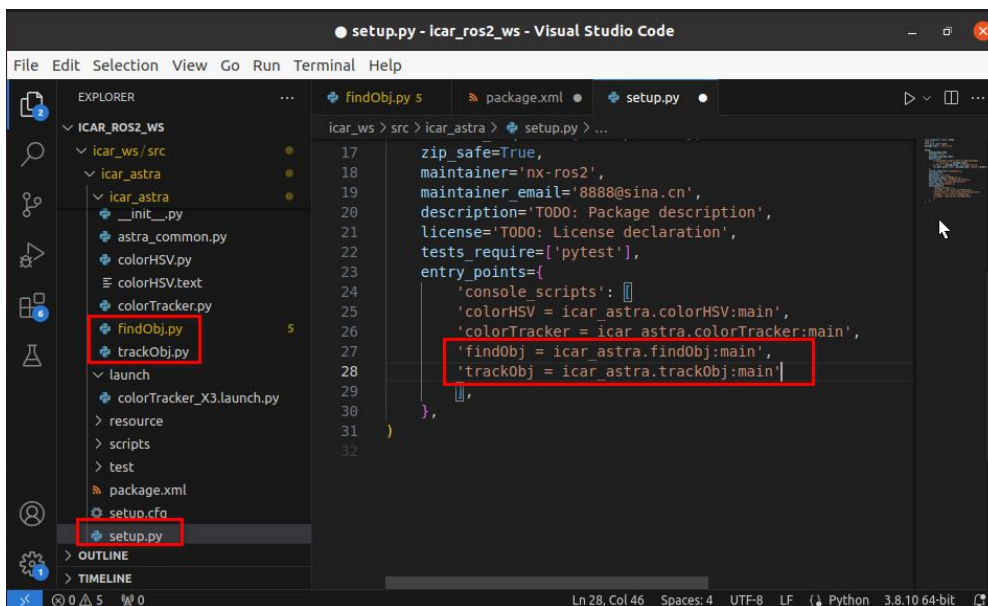
Activities  Terminal  9月 10 17:43  10W
root@jetson-desktop: ~/icar_ros2_ws/icar_ws/src

fectionate_leakey
jetson@jetson-desktop:~$ docker exec -it e49 /bin/bash
Error response from daemon: Container e49c1206ae0f7d19051bed4745e10616015c56b4378fdcd4815982cbb548c is not running
jetson@jetson-desktop:~$ docker start e49
e49
jetson@jetson-desktop:~$ docker exec -it e49 /bin/bash
-----
ROS_DOMAIN_ID: 32
my_robot_type: x3 | my_lidar: a1 | my_camera: astraplus
-----
root@jetson-desktop:~$ pwd
/
root@jetson-desktop:~$ ls
CMakeLists.txt  README.md  core  etc  log  opt  root  sbin  sys  var
CMakeModules  bin  dev  home  media  package.xml  ros_entrypoint.sh  src  tmp
LICENSE  boot  docker  lib  mnt  proc  run  srv  usr

root@jetson-desktop:~$ src
root@jetson-desktop:~/icar_ros2_ws/icar_ws/src# pwd
/root/icar_ros2_ws/icar_ws/src
root@jetson-desktop:~/icar_ros2_ws/icar_ws/src# ls
build          icar_description  icar_nav          laserscan_to_point_publisher
icar_KCFTracker  icar_description_x1  icar_point        log
icar_app_save_map  icar_laser          icar_slam          robot_pose_publisher
icar_astra        icar_linefollow     icar_visual        robot_pose_publisher_ros2
icar_base_node    icar_mediatpipe     icar_voice_ctrl
icar_bringup      icar_msgs           icar_web_savmap_interfaces
icar_ctrl         icar_multi          install
root@jetson-desktop:~/icar_ros2_ws/icar_ws/src#

```

将代码上传到 docker 中替换以后，还需要修改如下文件 setup.py，增加调用点：



```

● setup.py - icar_ros2_ws - Visual Studio Code
File Edit Selection View Go Run Terminal Help
EXPLORER  icar_ws > src > icar_astra > setup.py > ...
  icar_ros2_ws
  icar_ws/src
    icar_astra
      __init__.py
      astra_common.py
      colorHSV.py
      colorHSV.text
      colorTracker.py
      findObj.py
      trackObj.py
    launch
      colorTracker_X3.launch.py
    resource
    scripts
    test
    package.xml
    setup.cfg
    setup.py
  OUTLINE
  TIMELINE

17 zip_safe=True,
18 maintainer='nx-ros2',
19 maintainer_email='8888@sina.cn',
20 description='TODO: Package description',
21 license='TODO: License declaration',
22 tests_require=['pytest'],
23 entry_points={
24     'console_scripts': [
25         'colorHSV = icar_astra.colorHSV:main',
26         'colorTracker = icar_astra.colorTracker:main',
27         'findObj = icar_astra.findObj:main',
28         'trackObj = icar_astra.trackObj:main'
29     ],
30 }
31 )
32
Ln 28, Col 46  Spaces: 4  UTF-8  LF  Python  3.8.10 64-bit

```

需在 docker 中的~/icar_ros2_ws/icar_ws 目录中执行指令，来重新编译生成新的目标

检测应用节点：

```
colcon build
```

8. 运行 ROS 目标检测应用：

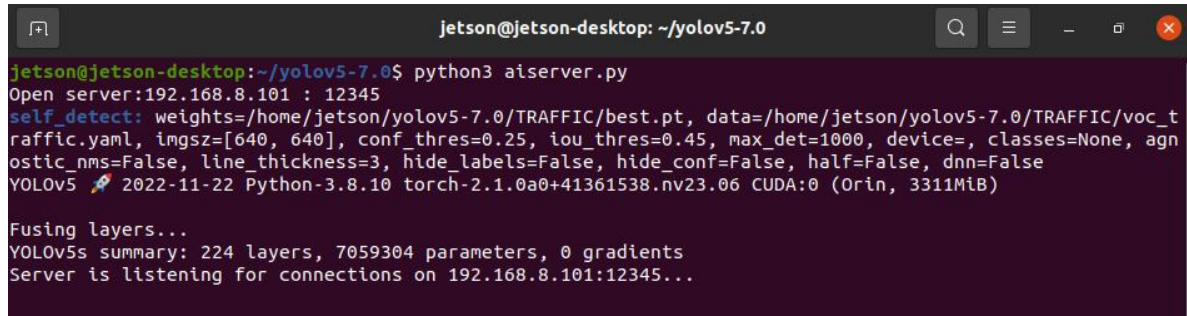
参考 6.4 章节的任务五，类似的方式，要打开四个 docker 环境 Terminal，按如下步骤

启动 ROS2 的目标检测应用。

(1) 实现使用如下指令来启动 AI 推理服务程序：

```
cd ~/yolov5-7.0

python3 aiserver.py
```



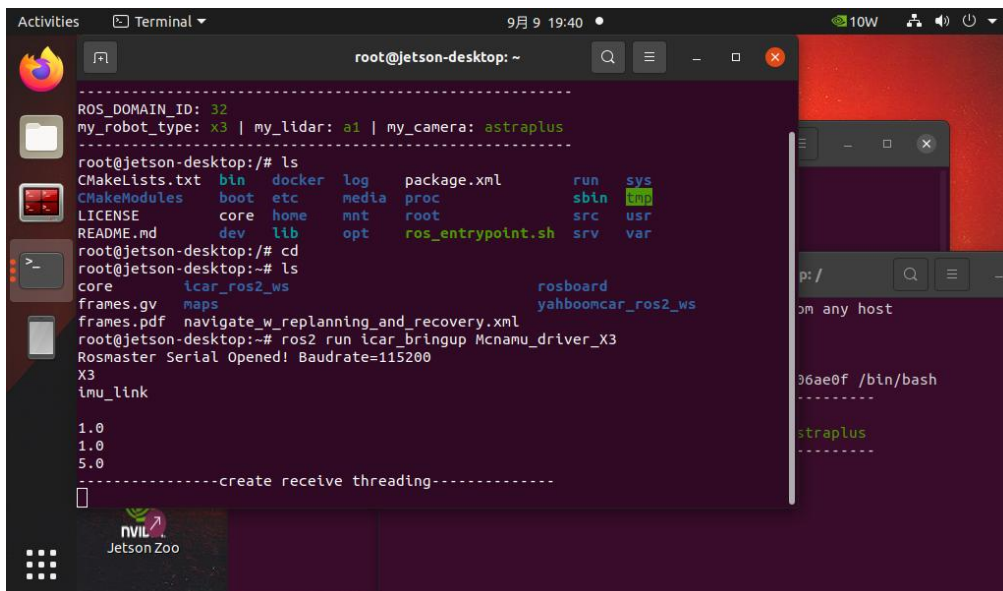
```
jetson@jetson-desktop: ~/yolov5-7.0
jetson@jetson-desktop:~/yolov5-7.0$ python3 aiserver.py
Open server:192.168.8.101 : 12345
self_detect: weights=/home/jetson/yolov5-7.0/TRAFFIC/best.pt, data=/home/jetson/yolov5-7.0/TRAFFIC/voc_t
raffic.yaml, imgsz=[640, 640], conf_thres=0.25, iou_thres=0.45, max_det=1000, device=, classes=None, agn
ostic_nms=False, line_thickness=3, hide_labels=False, hide_conf=False, half=False, dnn=False
YOLOv5 🚀 2022-11-22 Python-3.8.10 torch-2.1.0a0+41361538.nv23.06 CUDA:0 (Orin, 3311MiB)

Fusing layers...
YOLOv5s summary: 224 layers, 7059304 parameters, 0 gradients
Server is listening for connections on 192.168.8.101:12345...
```

从打屏信息可以看到，服务端已经在 IP 地址 192.168.8.101 上开启了 12345 端口的监
听，等待客户端连接。

(2) 在一个 docker 终端中启动 ROS2 底盘控制程序。

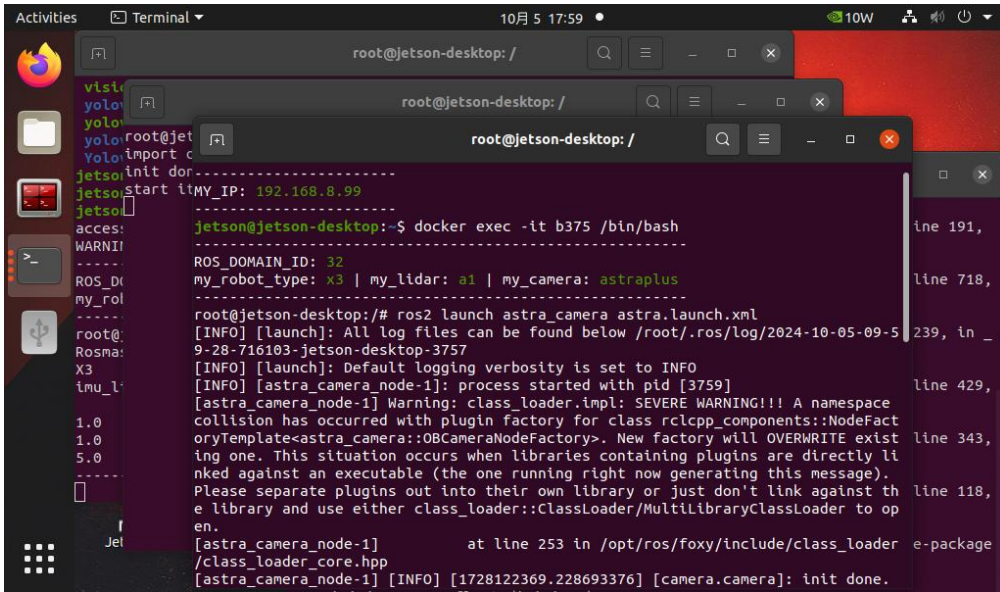
```
ros2 run icar_bringup Mcnamu_driver_X3
```



```
root@jetson-desktop: ~
ROS_DOMAIN_ID: 32
my_robot_type: x3 | my_lidar: a1 | my_camera: astraplus
root@jetson-desktop:~/# ls
CMakeLists.txt  bin  docker  log  package.xml  run  sys
CMakeModules  boot  etc  media  proc  sbin  top
LICENSE  core  home  mnt  root  src  usr
README.md  dev  lib  opt  ros_entrypoint.sh  srv  var
root@jetson-desktop:~/# cd
root@jetson-desktop:~/# ls
core  icar_ros2_ws  rosboard
frames.gv  maps  yahboomcar_ros2_ws
frames.pdf  navigate_w_replanning_and_recovery.xml
root@jetson-desktop:~/# ros2 run icar_bringup Mcnamu_driver_X3
Rosmaster Serial Opened! Baudrate=115200
X3
imu_link
1.0
1.0
5.0
-----create receive threading-----
```

(3) 在第二个终端中启动 ros2 深度相机应用程序。

```
ros2 launch astra_camera astra.launch.xml
```



(4) 在第三个终端中启动 ros2 目标检测应用程序。

```
ros2 run icar_astra findObj
```

```
root@jetson-desktop: /51x13
root@jetson-desktop: /# ros2 run icar_astra findObj
import finish
cv_edition: 4.7.0
start it
a1
a2
Result: ['167.5', '355.5000114440918', 'turnleft',
'1731595323.771113']
a1
a2
Result: ['167.5', '355.5000114440918', 'turnleft',
'1731595325.111005']
a1
a2
```

(5) 在第四个终端中启动 ros2 目标追踪应用程序

```
ros2 run icar_astra trackObj
```

```
root@jetson-desktop: / 50x13
-----
root@jetson-desktop: /# ros2 run icar_astra track0b
j
import done
init done
start it
twist.linear.x: 0.0
twist.angular.z: 2.0
twist.linear.x: 0.0
twist.angular.z: 0.765
twist.linear.x: 0.0
twist.angular.z: 0.765
twist.linear.x: 0.0
```


在 iCar 智能小车前方摆放目标检测物（交通标识图像纸板等），此时，拔掉键盘鼠标，将 iCar 服务机器从四轮悬空的状态放到地面上，iCar 智能小车将开启目标检测识别追踪，并选择相机视野内符合条件的目标对象进行追踪，iCar 智能小车盯住目标对象的方向转动朝向，并根据距离远近来进行前进或者后退，保持和目标对象的距离在一个预设范围。

1. 了解了目标检测神经网络 YOLOv5 及其特点。
2. 了解了 YOLOv5 模型训练软硬件环境需求及搭建过程。
3. 掌握了 Anaconda 的安装过程及创建虚拟环境的过程。
4. 掌握了 YOLOv5 的下载及安装依赖的过程。
5. 掌握了对 YOLOv5 标注数据集进行拆分的方法。
6. 掌握了如何配置 YOLOv5 训练参数。

7. 掌握了如何进行 YOLOv5 训练，并对模型效果进行评估。

8

实验资源释放

1. 关闭终端窗口。
2. 关闭宿主机。

9

实验小结

通过准备 YOLOv5 模型训练的软硬件环境，让开发者了解到进行 YOLOv5 模型训练所需要的硬件设备、软件环境、yolov5-7.0 版本的下载及依赖安装过程，并通过数据集拆分让开发者了解到拆分方法和不同数据集的用途以及配置方法，通过 YOLOv5 模型训练让开发者了解到如何进行模型训练以及对训练结果进行效果评估。

该实验为下一步在 ROS 中开发目标检测应用打下基础，ROS 只需要在边缘端设备上调用本节训练出的 YOLOv5 模型即可进行目标检测。